

Framework Explanation Document

BlazeDemo Automation Framework

Hybrid Framework: Page Object Model + Data-Driven Testing

SDET TRAINING PROGRAM

Prepared By: Nishant Kumar

GitHub: https://github.com/NishantKumar7007/WIPRO_Capstone.git

Field	Details
Application Under Test	Simple Travel Agency - https://blazedemo.com/
Automation Goal	Automate end-to-end flight booking flow
Framework Type	Hybrid Framework (POM + Data-Driven)
Tools	Java, Selenium WebDriver, TestNG, Maven, Git
Test Data	Excel/CSV based test data
Output	Booking confirmation message and report generation

1. Objective of the Capstone

The goal of this capstone project is to build a real-time Selenium automation framework for the BlazeDemo flight booking application.

- Build a Selenium Automation Framework using Java.
- Automate the complete end-to-end booking flow.
- Apply Java, Selenium WebDriver, TestNG, and Maven concepts.
- Understand industry-level testing practices.
- Create a reusable and maintainable automation structure.

2. Learning Outcomes

After completing this project, learners should be able to:

- Design and implement Page Object Model (POM).
- Implement data-driven testing using Excel or CSV.
- Handle dropdowns, buttons, forms, and dynamic web elements.
- Use TestNG annotations, assertions, and DataProvider.
- Generate automation reports after execution.
- Follow clean coding practices and maintain a proper framework structure.

3. Tools and Technologies Used

Tool / Technology	Purpose
Java	Programming language used to write automation scripts.

Selenium WebDriver	Used to automate browser actions.
TestNG	Used for test execution, assertions, annotations, and reporting.
Maven	Used for dependency management and build execution.
Page Object Model	Used to separate page locators/actions from test logic.
Git	Used for version control and code sharing.
Excel/CSV	Used to maintain test data for data-driven testing.
Jenkins	Optional CI/CD tool used to run tests automatically.

4. Framework Type and Requirements

This project uses a Hybrid Framework. It combines Page Object Model and Data-Driven Testing.

- POM keeps locators and page actions in separate page classes.
- Data-driven testing allows the same test case to run with multiple sets of test data.
- TestNG controls the test execution flow using annotations and assertions.
- Maven manages project dependencies and test execution commands.

5. Framework Folder Structure

```

BlazeDemo_Capstone
├── src/test/java
│   ├── base
│   │   └── BaseClass.java
│   ├── pages
│   │   ├── HomePage.java
│   │   ├── FlightsPage.java
│   │   ├── PurchasePage.java
│   │   └── ConfirmationPage.java
│   ├── tests
│   │   └── FlightBookingTest.java
│   ├── utils
│   │   ├── ExcelUtils.java
│   │   └── ExtentReportManager.java
│   └── listeners
│       └── TestListener.java
├── src/test/resources
│   ├── config.properties
│   └── testdata.xlsx
├── pom.xml
└── testng.xml

```

6. Framework Components

Component	Explanation
Base Class	Contains WebDriver setup, browser launch, waits, and browser close logic.
Page Classes	Contains locators and reusable methods for each web page.
Test Classes	Contains test cases and verification steps using TestNG.
Utility Classes	Contains common reusable code such as Excel reading, screenshots, waits, and reports.
Config File	Stores common values like browser name, application URL, and timeout.
Test Data File	Stores passenger and payment details used during booking.

Reports	Stores execution result with pass/fail status.
Listeners	Captures screenshots or logs automatically when a test fails.

7. Problem Statement

Automate the end-to-end flight booking flow on the Simple Travel Agency application: <https://blazedemo.com/>

1. Navigate to the Simple Travel Agency application.
2. Select a departure city from the dropdown.
3. Select a destination city from the dropdown.
4. Click on the Find Flights button.
5. Verify that a list of available flights is displayed.
6. Select one flight by clicking Choose This Flight.
7. Verify that the Purchase Flight page is displayed with flight details and total cost.
8. Enter valid personal details required for booking.
9. Enter valid payment details.
10. Click on the Purchase Flight button.
11. Verify that the booking is completed successfully and a confirmation message is displayed.

8. Automation Test Flow

Launch Browser -> Open BlazeDemo -> Select Cities -> Find Flights -> Verify Flight List -> Choose Flight -> Verify Purchase Page -> Enter Details -> Purchase Flight -> Verify Confirmation

9. Input Constraints

- Departure and destination cities must be selected before searching for flights.
- All mandatory personal and payment fields must be filled.
- Only one flight should be selected per execution.
- Valid test data should be available in Excel or CSV file.

10. Expected Output

- Flight search result page should display available flights.
- Purchase page should display selected flight details and total cost.
- After clicking Purchase Flight, confirmation page should be displayed.
- Automation report should show the execution status as Passed when booking is completed successfully.

11. TestNG Usage

- @BeforeMethod is used to launch the browser and open the application.
- @Test is used to write the booking test case.
- @DataProvider is used to run the test with multiple data rows.
- Assert is used to verify flight list, purchase page, and confirmation page.
- @AfterMethod is used to close the browser after test execution.

12. Reporting and Screenshots

- Extent Report can be generated after test execution.
- Report contains test name, status, steps, and failure details.
- Screenshots can be captured automatically for failed test cases.
- Reports help in debugging and project submission.

13. Execution Process

- Open the project in Eclipse.
- Make sure Maven dependencies are downloaded.
- Add valid data in the Excel or CSV test data file.
- Run testng.xml or execute Maven command: `mvn clean test`.
- Check console output and automation report after execution.

14. Jenkins CI/CD Pipeline Execution

The automation framework is integrated with Jenkins for Continuous Integration. The pipeline performs source code checkout, build, test execution, report archiving, and post-build actions.

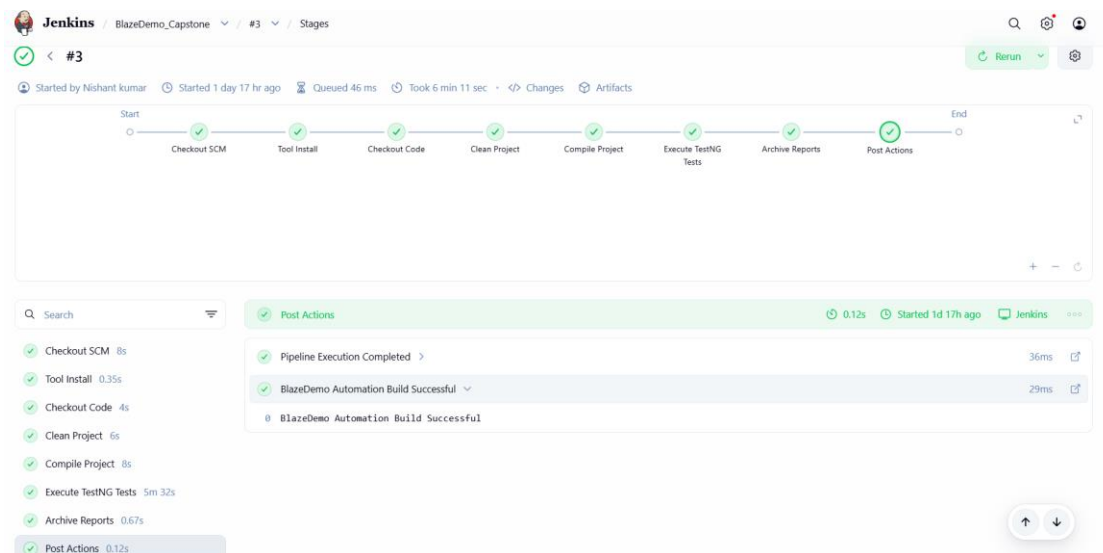


Figure: Successful Jenkins Pipeline Execution for BlazeDemo Automation Framework.

 Jenkins

BlazeDemo_Capstone

#3

Pipeline Steps







Status

Changes

Console Output

Edit Build Information

Delete build '#3'

Timings

Git Build Data

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

Step	Arguments	Status
Start of Pipeline - (6 min 3 sec in block)		
node - (6 min 2 sec in block)		
node block - (6 min 2 sec in block)		
stage - (8.1 sec in block)	Declarative: Checkout SCM	
stage block (Declarative: Checkout SCM) - (8 sec in block)		
checkout - (7.9 sec in self)		
withEnv - (5 min 53 sec in block)	GIT_BRANCH, GIT_COMMIT, GIT_PREVIOUS_COMMIT, GIT_PREVIOUS_SUCCESSFUL_COMMIT, GIT_URL	
withEnv block - (5 min 53 sec in block)		
stage - (0.46 sec in block)	Declarative: Tool Install	
stage block (Declarative: Tool Install) - (0.35 sec in block)		
tool - (65 ms in self)	JDK	
envVarsForTool - (93 ms in self)		
tool - (57 ms in self)	Maven	
envVarsForTool - (69 ms in self)		

 Jenkins			BlazeDemo_Capstone	▼	#3	▼	Pipeline Steps	  
			withEnv block - (5 min 53 sec in block)					✓
			stage - (4.1 sec in block)	Checkout Code				✓
			stage block (Checkout Code) - (4 sec in block)					✓
			tool - (68 ms in self)	JDK				✓
			envVarsForTool - (77 ms in self)					✓
			tool - (62 ms in self)	Maven				✓
			envVarsForTool - (83 ms in self)					✓
			withEnv - (3.6 sec in block)	JAVA_HOME, PATH+JDK, M2_HOME, MAVEN_HOME, PATH+MAVEN				✓
			withEnv block - (3.5 sec in block)					✓
			git - (3.5 sec in self)					✓
			stage - (6.8 sec in block)	Clean Project				✓
			stage block (Clean Project) - (6.7 sec in block)					✓
			tool - (59 ms in self)	JDK				✓
			envVarsForTool - (78 ms in self)					✓
			tool - (59 ms in self)	Maven				✓
			stage - (0.01 sec in block)	Archive Reports				✓
			stage block (Archive Reports) - (0.67 sec in block)					✓
			tool - (44 ms in self)	JDK				✓
			envVarsForTool - (53 ms in self)					✓
			tool - (44 ms in self)	Maven				✓
			envVarsForTool - (56 ms in self)					✓
			withEnv - (0.38 sec in block)	JAVA_HOME, PATH+JDK, M2_HOME, MAVEN_HOME, PATH+MAVEN				✓
			withEnv block - (0.32 sec in block)					✓
			archiveArtifacts - (0.15 sec in self)					✓
			archiveArtifacts - (82 ms in self)					✓
			archiveArtifacts - (46 ms in self)					✓
			stage - (0.21 sec in block)	Declarative: Post Actions				✓
			stage block (Declarative: Post Actions) - (0.12 sec in block)					✓
			echo - (36 ms in self)	Pipeline Execution Completed				✓
			echo - (29 ms in self)	BlazeDemo Automation Build Successful				✓


Jenkins / BlazeDemo_Capstone / #3

```

Flight booking completed successfully
PASSED : verifyEndToEndFlightBooking
=== Test Execution Finished ===
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 318.4 s -- in TestSuite
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 05:27 min
[INFO] Finished at: 2026-06-11T16:46:12+05:30
[INFO] -----
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Archive Reports)
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] withEnv
[Pipeline] {
[Pipeline] archiveArtifacts
Archiving artifacts
[Pipeline] archiveArtifacts
Archiving artifacts

[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] withEnv
[Pipeline] {
[Pipeline] archiveArtifacts
Archiving artifacts
[Pipeline] archiveArtifacts
Archiving artifacts
[Pipeline] archiveArtifacts
Archiving artifacts
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline Execution Completed
[Pipeline] echo
BlazeDemo Automation Build Successful
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

localhost:8080


Jenkins / BlazeDemo_Capstone / #3

15. Conclusion

This BlazeDemo automation project demonstrates an industry-style hybrid automation framework using Java, Selenium WebDriver, TestNG, Maven, POM, and data-driven testing. The framework is reusable, maintainable, easy to understand, and suitable for capstone project submission.

